

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И
ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № N3
по дисциплине «Информационные технологии и программирование»
по теме: «Разработка веб-приложения на Spring Boot»

Выполнил студент группы:
БПИ2401 Исламов Эмин Маратович
Проверил: Харрасов Камиль
Раисович

Москва 2026

Цель работы

Научиться создавать Spring Boot приложение, освоить принципы инверсии управления (IoC) и внедрения зависимостей (DI), изучить три способа конфигурации Spring-компонентов, а также научиться разрешать конфликты при наличии нескольких бинов одного типа.

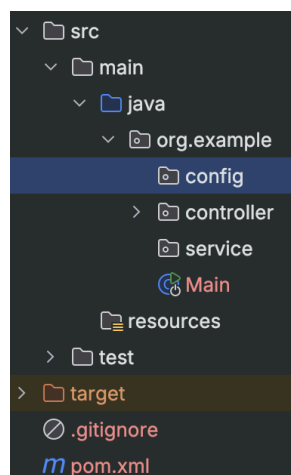
Ход работы

Часть 0. Создание проекта и установка зависимостей.

В `pom.xml` добавляем зависимости:

```
<dependencies>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <version>3.4.3</version>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <version>3.4.3</version>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
</dependencies>
```

Структура проекта:



Часть 1. Знакомство с Spring Boot Web

/controller/HelloController.java

```
package org.example.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello Spring Boot!";
    }

    @GetMapping("/goodbye")
    public String goodbye() {
        return "Goodbye Spring Boot!";
    }

    @GetMapping("/greet")
    public String greet(@RequestParam("name") String name) {
        return "Greeting " + name;
    }

    @GetMapping("/info")
    public String info(@RequestParam("name") String name,
        @RequestParam("age") int age) {
        return "Hello " + name + " " + age;
    }
}
```

Запуск приложения в Main.java

```
package org.example;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Main {

    public static void main(String[] args) {
        SpringApplication.run(Main.class, args);
    }
}
```

Проверка:

```
emin@MacBook-Air-Emin spring-lab3-notifications % curl "http://localhost:8080/hello"
Hello Spring Boot!%
emin@MacBook-Air-Emin spring-lab3-notifications % curl "http://localhost:8080/goodbye"
Goodbye Spring Boot!%
emin@MacBook-Air-Emin spring-lab3-notifications % curl "http://localhost:8080/greet?
name=Ivan"
Greeting Ivan%
emin@MacBook-Air-Emin spring-lab3-notifications % curl "http://localhost:8080/info?
name=Ivan&age=20"
Hello Ivan 20%
emin@MacBook-Air-Emin spring-lab3-notifications %
```

Часть 2. Проблема жестких связей.

Интерфейс MessageService.java

```
package org.example.service;

public interface MessageService {

    void sendMessage(String message, String recipient);

}
```

Реализации:

EmailService.java

```
package org.example.service;

public class EmailService implements MessageService {

    @Override

    public void sendMessage(String message, String recipient) {

        System.out.println("Email sent to " + recipient + ": " +
message);

    }

}
```

SmsService.java

```
package org.example.service;

public class SmsService implements MessageService {

    @Override

    public void sendMessage(String message, String recipient) {

        System.out.println("SMS sent to " + recipient + ": " +
message);

    }

}
```

PushService.java

```
package org.example.service;

public class PushService implements MessageService {

    @Override
    public void sendMessage(String message, String recipient){
        System.out.println("Push to " + recipient + " : " + message);
    }
}
```

NotificationManager.java

```
package org.example.service;

public class NotificationManager {

    private final MessageService messageService;

    public NotificationManager(MessageService messageService) {
        this.messageService = messageService;
    }

    public void notify(String message, String recipient) {
        messageService.sendMessage(message, recipient);
    }
}
```

Контроллер NotificationController.java

```
@RestController

public class NotificationController {

    @GetMapping("/notify")
    public String notify(
        @RequestParam("message") String message,
        @RequestParam("recipient") String recipient,
        @RequestParam("type") String type) {
        MessageService service = switch (type.toLowerCase()) {
            case "sms" -> new SmsService();
            case "push" -> new PushService();
            default -> new EmailService();
        };

        NotificationManager manager = new
        NotificationManager(service);
    }
}
```

```

        manager.notify(message, recipient);

        return "Notification sent: type=" + type + ", message=" +
message + ", recipient=" + recipient;
    }
}

```

Проверка:

```

emin@MacBook-Air-Emin spring-lab3-notifications % curl "http://localhost:8080/notify?
message=Hi&recipient=test@test.com&type=email"
Notification sent: type=email, message=Hi, recipient=test@test.com%
emin@MacBook-Air-Emin spring-lab3-notifications % curl "http://localhost:8080/notify?
message=Hi&recipient=79990001122&type=sms"
Notification sent: type=sms, message=Hi, recipient=79990001122%
emin@MacBook-Air-Emin spring-lab3-notifications % curl "http://localhost:8080/notify?
message=Hi&recipient=@emin&type=push"
Notification sent: type=push, message=Hi, recipient=@emin%
emin@MacBook-Air-Emin spring-lab3-notifications %

```

Проблемы: выбор реализации делается вручную, код контроллера разрастается; сложно централизованно управлять жизненным циклом объектов; тестирование, замена зависимостей растёт, а тестирование усложняется при увеличении числа сервисов.

Часть 3. Java Config: @Configuration + @Bean

AppConfig.java

```

@Configuration
public class AppConfig {

    @Bean
    public EmailService emailService() {
        return new EmailService();
    }

    @Bean
    public PushService pushService() {
        return new PushService();
    }

    @Bean
    public NotificationManager notificationManager(SmsService
smsService) {

```

```

        //SmsService теперь приходит из AnotherConfig как бин
        return new NotificationManager(smsService);
    }
}

```

ANotherConfig.java

```

@Configuration
public class AnotherConfig {

    @Bean
    public SmsService smsService() {

        return new SmsService();
    }
}

```

NotificationManager.java

```

public class NotificationManager {

    private final MessageService messageService;

    public NotificationManager(MessageService messageService) {

        this.messageService = messageService;
    }

    public void notify(String message, String recipient) {

        messageService.sendMessage(message, recipient);
    }
}

```

Часть 4. Аннотации @ComponentScan @Service @Autowired

EmailService.java

```

@Service
public class EmailService implements MessageService {

    @Override
    public void sendMessage(String message, String recipient) {

        System.out.println("Email sent to " + recipient + ": " +
message);
    }
}

```

```
}  
}
```

SmsService.java аналогчно

NotificationManager.java

```
@Service  
public class NotificationManager {  
    private final MessageService messageService;  
    public NotificationManager(MessageService  
messageService) {  
        this.messageService = messageService;  
    }  
    public void notify(String message, String recipient) {  
        messageService.sendMessage(message, recipient);  
    }  
}
```

Уже на этом этапе IDE предупреждает нас, что автопривязка невозможна, так как больше одного Бина типа MessageService. На данном этапе можно пофиксить добавлением квалифаера @Qualifier(«») или отметить конкретный бин приоритетным @Primary.

Упрощаем NotificationManager.java

```
@RestController  
public class NotificationController {  
    private final NotificationManager notificationManager;  
    public NotificationController(NotificationManager  
notificationManager) {  
        this.notificationManager = notificationManager;  
    }  
    @GetMapping("/notify")  
    public String notify(@RequestParam("message") String message,  
@RequestParam("email") String email) {
```



```

        notificationManager.notify(message, email);
        return "Уведомление отправлено через Java Config";
    }
}

```

AppConfig.java также упрощается

```

@Configuration
@ComponentScan("org.example")
public class AppConfig {}

```

Вывод: ошибка возникает потому что Spring при внедрении зависимостей пытается внедрить зависимость MessageService, а в контексте несколько Бинов этого типа. Спрингу необходимо выбрать один бин, но кандидатов несколько.

Если убрать @Service с одного сервиса, например, с SmsService, то он перестанет быть бином, он не регистрируется при сканировании. Ошибка неоднозначности исчезнет и Спринг сможет однозначно внедрить другой сервис.

Без аннотаций Service, Component, Controller классы невидимы Спрингу.

Важно: если добавить @ComponentScan("org.example.controller") в AppConfig, то ошибка не возникнет и Спринг будет сканировать только контроллер и его подпакеты, СЕРВИСЫ НЕ ПОПАДУТ В СКАН. Как итог - приложение упадёт при старте с ошибкой вида No qualifying bean of type NotificationManager или NoSuchBeanDefinitionException.

Создадим AdvancedMessageService

```

public interface AdvancedMessageService extends MessageService {
    String getServiceType();
}

```

Реализуем в SmsService

```
@Service
public class SmsService implements AdvancedMessageService {
    @Override
    public void sendMessage(String message, String recipient) {
        System.out.println("SMS to " + recipient + ": " + message);
    }
    @Override
    public String getServiceType() {
        return "sms";
    }
}
```

Если нотификейшнМенеджер будет ожидать МесседжСервис, то неоднозначность не исчезнет, тк СмсСервис по прежнему является МесседжСервисом.

Если изменить нотификейшнМенеджер так, чтобы он ожидал ПродвинутыйМессджСервис, то Спринг внедрит только тот, кто им является. Конфликт исчезнет.

Часть 5. Разрешение конфликта бинов

1) Имеется несколько бинов MessageService. Помечаем только один как основной. SmsService

```
@Service
@Primary
public class SmsService implements MessageService {
    @Override
    public void sendMessage(String message, String
recipient) {
        System.out.println("SMS to " + recipient + ": " +
message);
    }
}
```

```

    }
}

```

2) @Qualifier: переименовать бин EmailService и исправить внедрение

```

@Service("customEmail")
public class EmailService implements MessageService {
    @Override
    public void sendMessage(String message, String
recipient) {
        System.out.println("EMAIL to " + recipient + ": " +
message);
    }
}

```

Тогда в менеджер внедряем с квалифаером на имя customEmail

3) Добавить TelegramService и внедрять список в менеджере

NotificationManager.java

```

@Service
public class NotificationManager {
    private final List<MessageService> messageServices;

    public NotificationManager(List<MessageService>
messageServices) {
        this.messageServices = messageServices;
    }

    public void notify(String message, String recipient) {
        messageServices.forEach(s -> s.sendMessage(message,
recipient));
    }
}

```

Spring будет собирать в список все Бины которые проходят по типу `MessageService`

4) Комбинирование `@Primary` и `List<MessageService>`

`@Primary` влияет только когда нужен один бин (`MessageService messageService`), а при внедрении коллекции (`List<MessageService>`) Spring отдаёт все бины, `@Primary` не фильтрует список

5) `Map<String, MessageService>` вместо `List` + выбор по параметру

Spring соберёт `map` так: ключ = имя бина (`emailService`, `smsService`, `customEmail`, `telegramService`, ...), а значение = объект сервиса.

`NotificationManager.java`

```
@Service
public class NotificationManager {
    private final Map<String, MessageService> services;

    public NotificationManager(Map<String, MessageService> services) {
        this.services = services;
    }

    // channels: например "sms,telegram" или "customEmail"
    public void notify(String message, String recipient, String
channels) {
        if (channels == null || channels.isBlank()) {
            throw new IllegalArgumentException("channels is empty");
        }
        for (String key : channels.split(",")) {
            String beanName = key.trim();
            MessageService service = services.get(beanName);
            if (service == null) {
                throw new IllegalArgumentException("Unknown channel: "
+ beanName +
                ". Available: " + services.keySet());
            }
            service.sendMessage(message, recipient);
        }
    }
}
```

Тогда контроллер должен передавать channels

```
@GetMapping("/notify")
public String notify(@RequestParam String message,
                    @RequestParam String recipient,
                    @RequestParam String channels) {
    notificationManager.notify(message, recipient,
    channels);
    return "OK";
}
```

Пример запроса:

/notify?message=Hi&recipient=test&channels=smsService,telegramService

Контрольные вопросы

1. @Component vs @Service vs @Repository vs @Controller

Все компоненты Spring, различие семантическое: Controller для web-слоя, Service для бизнес-логики, Repository для доступа к данным (может добавлять перевод исключений), Component это общий случай.

2. Как работает @ComponentScan?

Сканирует указанный пакет и под-пакеты, регистрирует найденные компоненты. Если указать неправильный пакет, то бины не найдутся.

3. Почему в Части 3 не было ошибки с двумя бинами, а в Части 4 появилась?

В Части 3 я явно создаёшь NotificationManager через @Bean и вручную выбираешь, какой сервис передать. В Части 4 Spring сам пытается внедрить MessageService и видит несколько кандидатов.

4. Способы разрешения конфликта бинов

@Primary, @Qualifier, внедрение List/Map всех бинов. Самый гибкий - это коллекция/карта.

5. Как внедрить все бины определённого типа?

List<MessageService> или Map<String, MessageService> в конструкторе.

6. Почему конструкторная DI лучше поля?

Неизменяемость, проще тестировать, зависимость обязательна, объект всегда в валидном состоянии.